

1 plateforme à concevoir
+ 2 architectes
= 3 possibilités

Raphaël SEMETEYS - Alexandre TOURET

Qui sommes-nous?

Raphaël SEMETEYS

Architecte entreprise

Developer advocate



@RaphaelSemeteys

https://dev.to/raphiki

Alexandre TOURET

Architecte logiciel

Developer advocate



@touret_alex

blog.touret.info

Who are we?

We design payments technology that powers the growth of millions of businesses around the world.



7000+ engineers
in over 40 countries



Managing 43+ billion
transactions per year

#1
European
payment
processor



€250M spent in R&D
every year



Handling 150+
payment methods

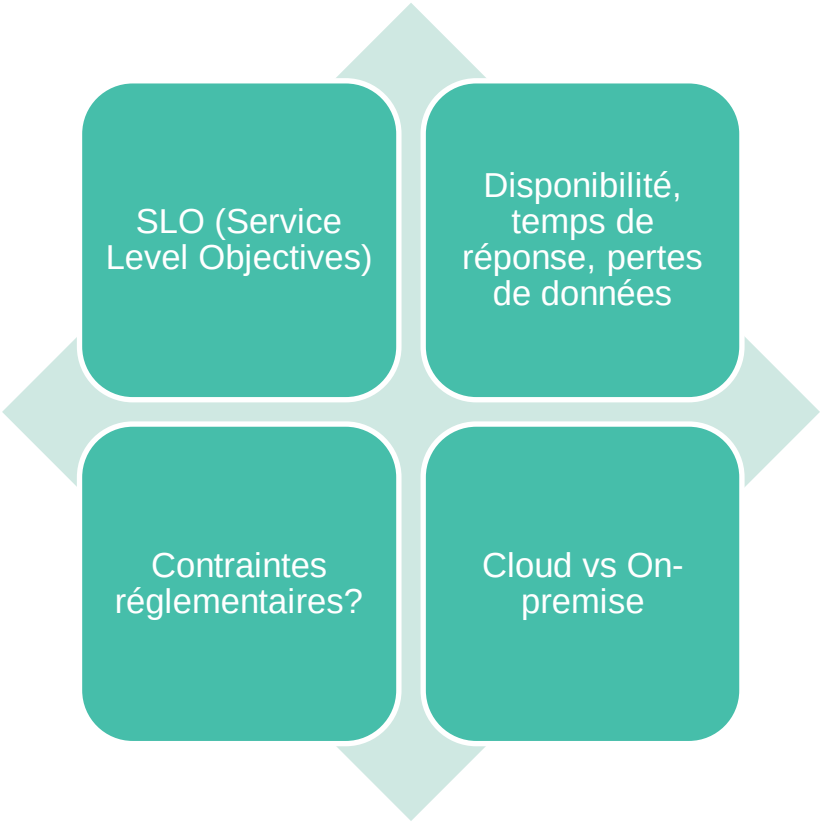
WORLDLINE


LE CREUSET

Vue métier



Exigences


SLO (Service Level Objectives)
Disponibilité, temps de réponse, pertes de données
Contraintes réglementaires?
Cloud vs On-premise

Architecture Decision Record

Référencez et stockez au plus près du code chaque décision structurante d'architecture

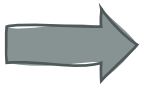
```
# Title

# Status
- [ ] proposed
- [X] accepted
- [ ] rejected
- [ ] deprecated
- [ ] superseded

# Context

# Decision

# Consequences
```



```
# ADR01 – Hébergement Cloud

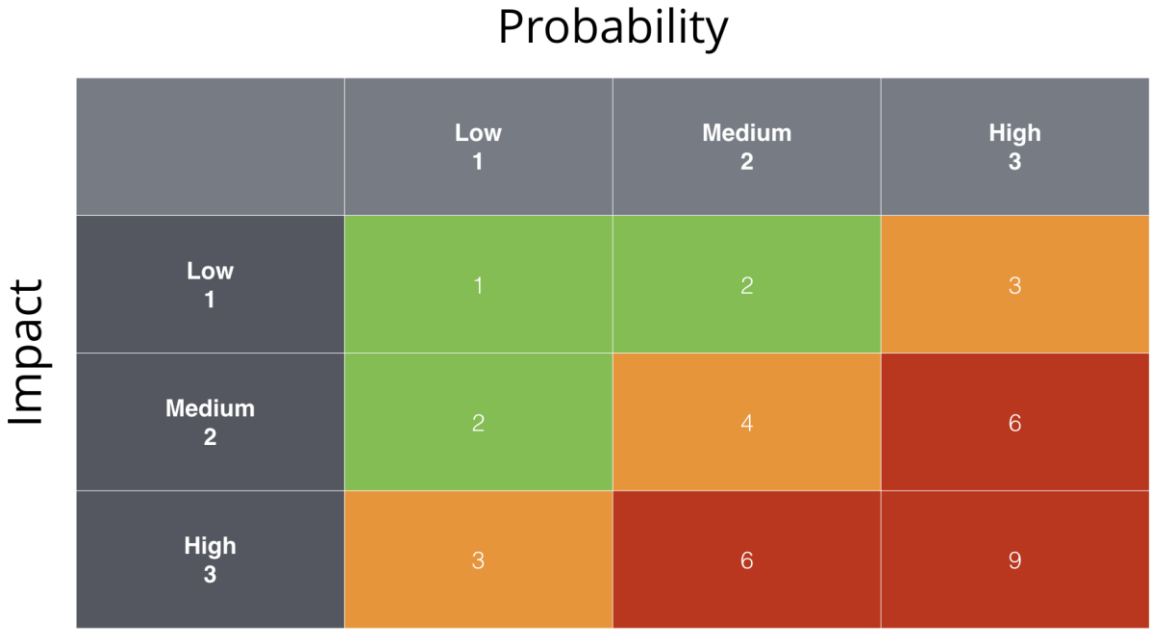
# Status
- [X] proposed
- [ ] accepted
- [ ] rejected
- [ ] deprecated
- [ ] superseded

# Context
La capacité de la plate-forme doit s'adapter en
fonction du succès de la nouvelle offre Truffade@Home

# Decision
Hébergement de type Cloud pour optimiser le coût à
l'usage et disposer de scalabilité intrinsèque

# Consequences
Disposer d'une architecture Cloud-native (12 factors)
```

Une analyse de risques ?



Source: riskstorming.com

Probability

Impact

	Low 1	Medium 2	High 3
Low 1			
Medium 2			
High 3	- Les middlewares indisponibles - Erreur d'accès à la database - Erreur SAN - Erreur réseau VLAN HS		

Vue métier : synthèse

Le besoin

- Une application de création et de livraison de Truffade à Domicile

Les exigences

- Retrieved Time Objective, Recovery Point Objective: ?
- Temps de réponse: 90% des transactions doivent être réalisées en moins de 2sec
- Disponibilité: 95%
- Nombre d'utilisateurs: cible métier à 500 000 / jour →*Peu de visibilité sur les pics*
- Capacité à intégrer facilement des nouveautés

Les contraintes réglementaires

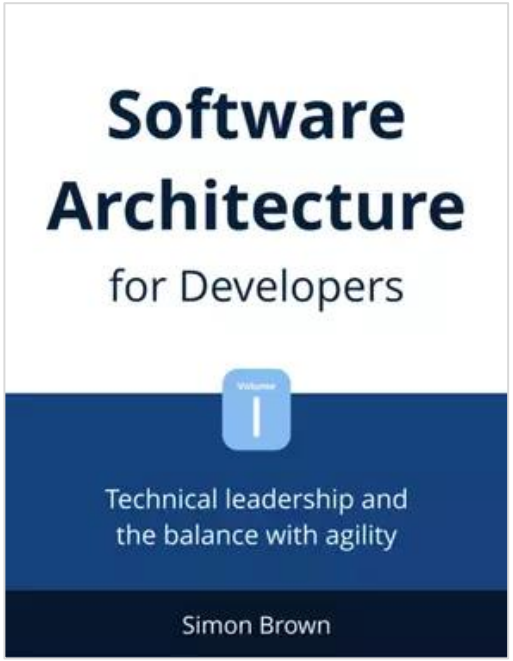
- Le paiement doit être conforme aux normes bancaires et paiement
- Le traitement des données doit être conforme au RGPD

Autres bonnes pratiques

- Tracer les décisions dans des ADRs, formaliser les risques à traiter (ou pas)
- Définir un vocabulaire métier commun (DDD)

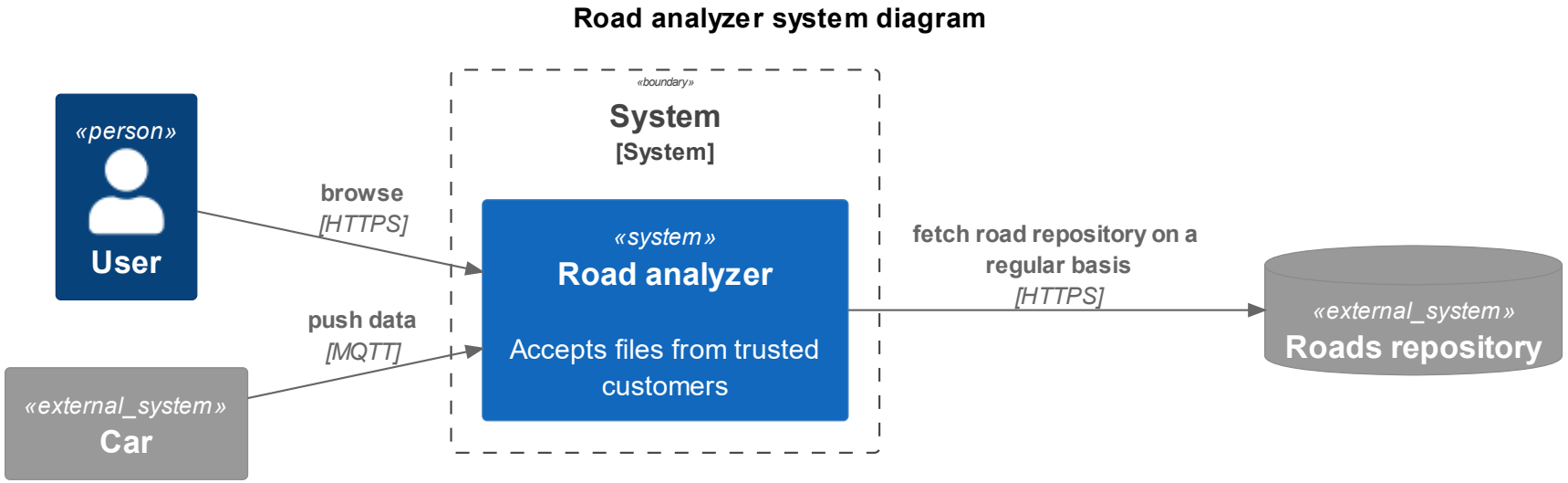
Démarche d’architecture

Modèle C4

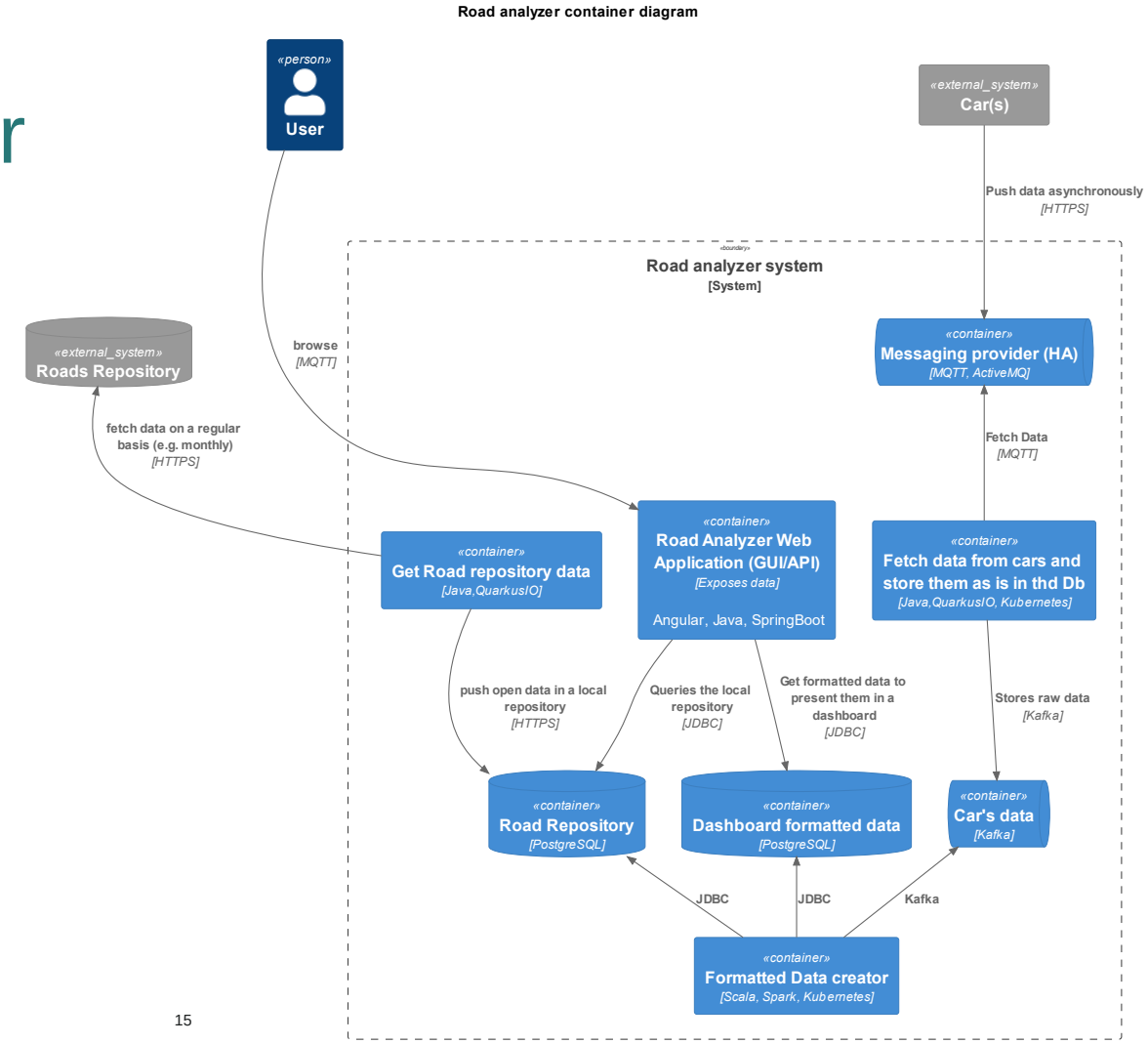


c4model.com

Vue C4 System

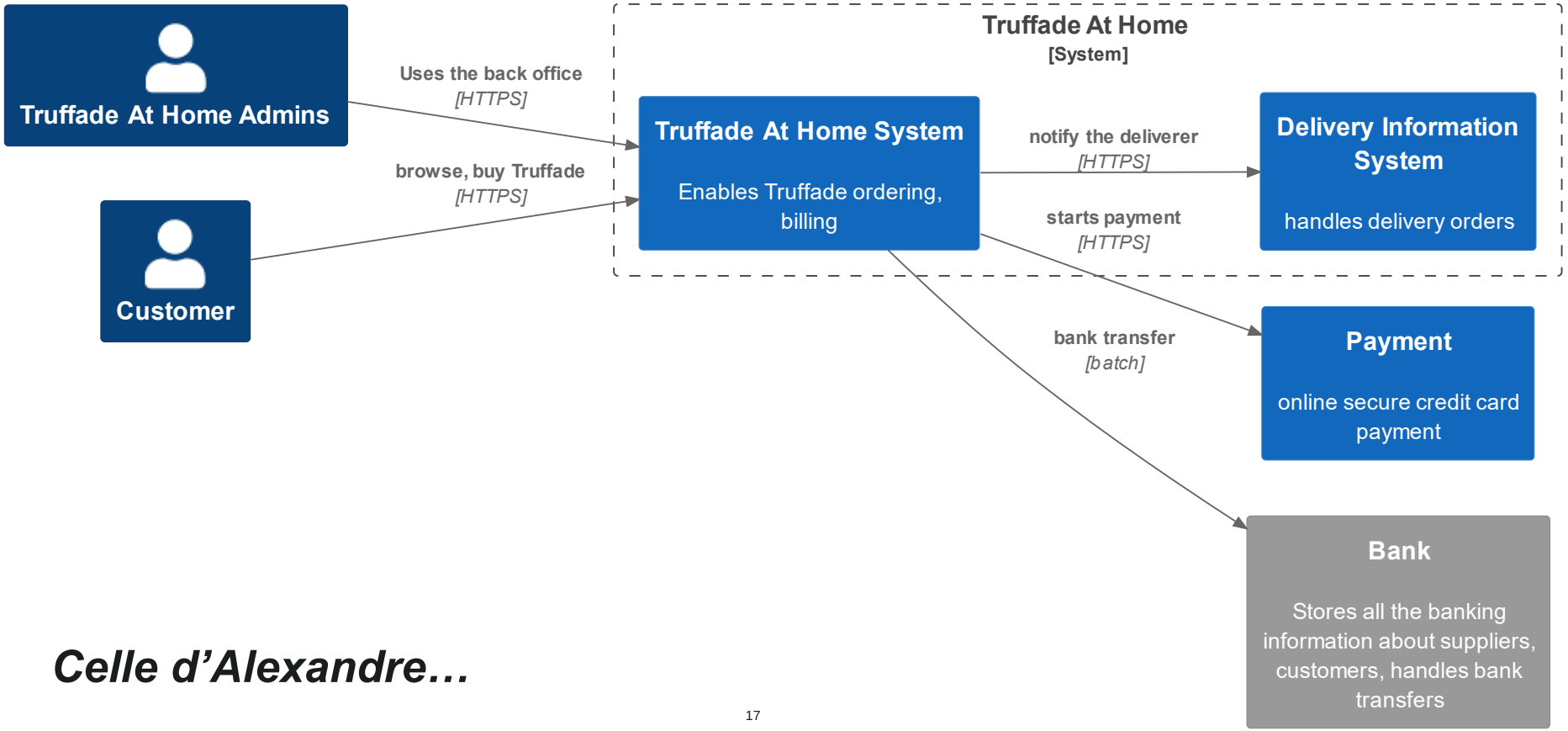


Vue C4 Container



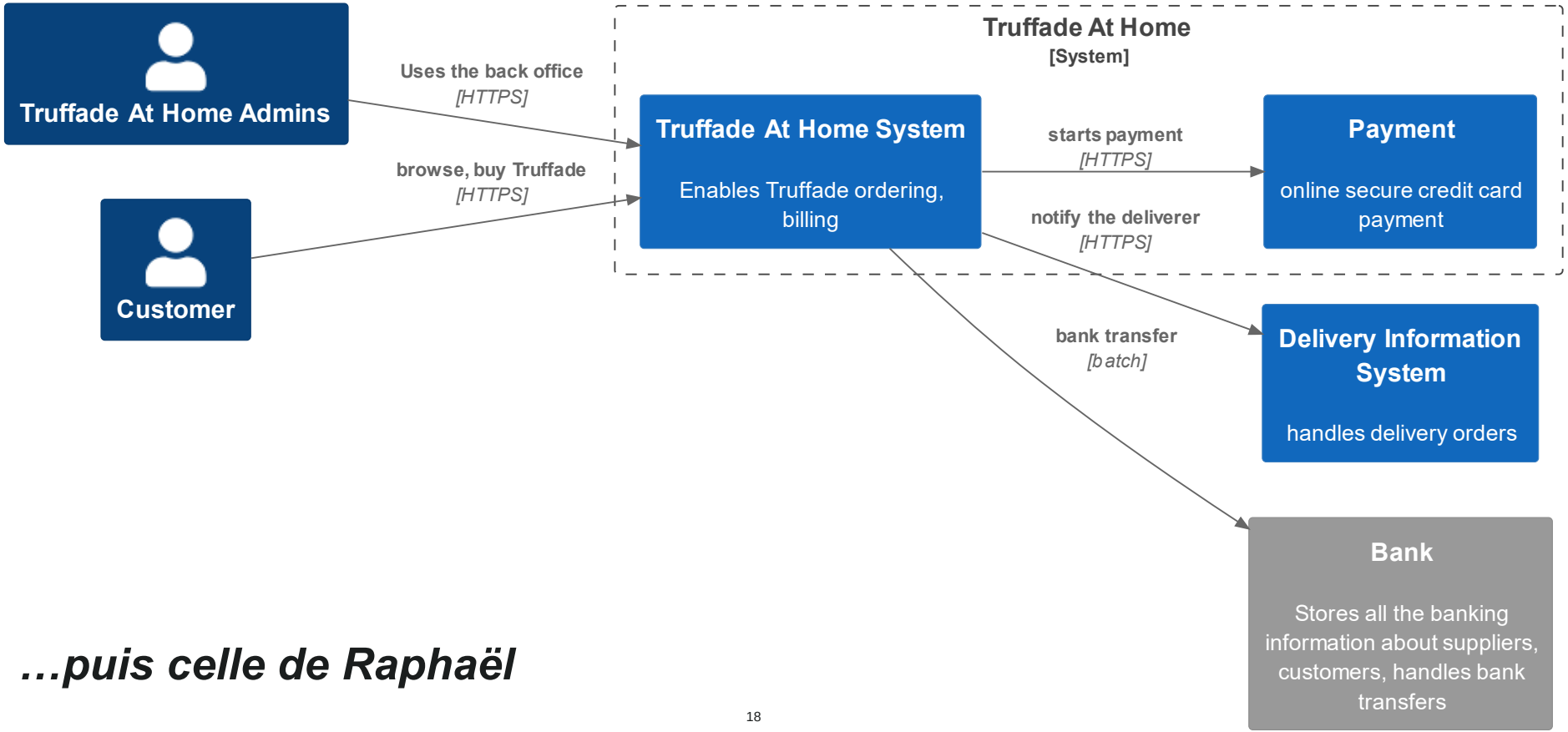
La vue fonctionnelle

Vue fonctionnelle



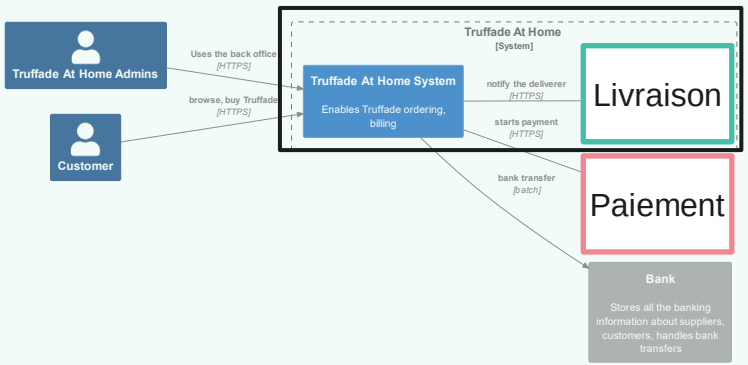
Celle d'Alexandre…

Vue fonctionnelle



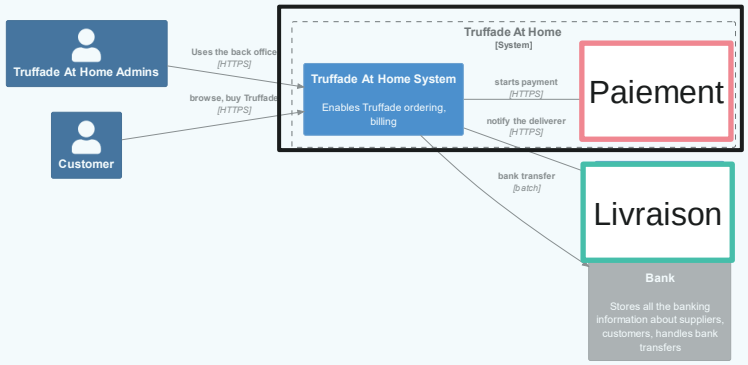
…puis celle de Raphaël

Quel est votre avis ?



1

La solution d'**Alexandre** ?

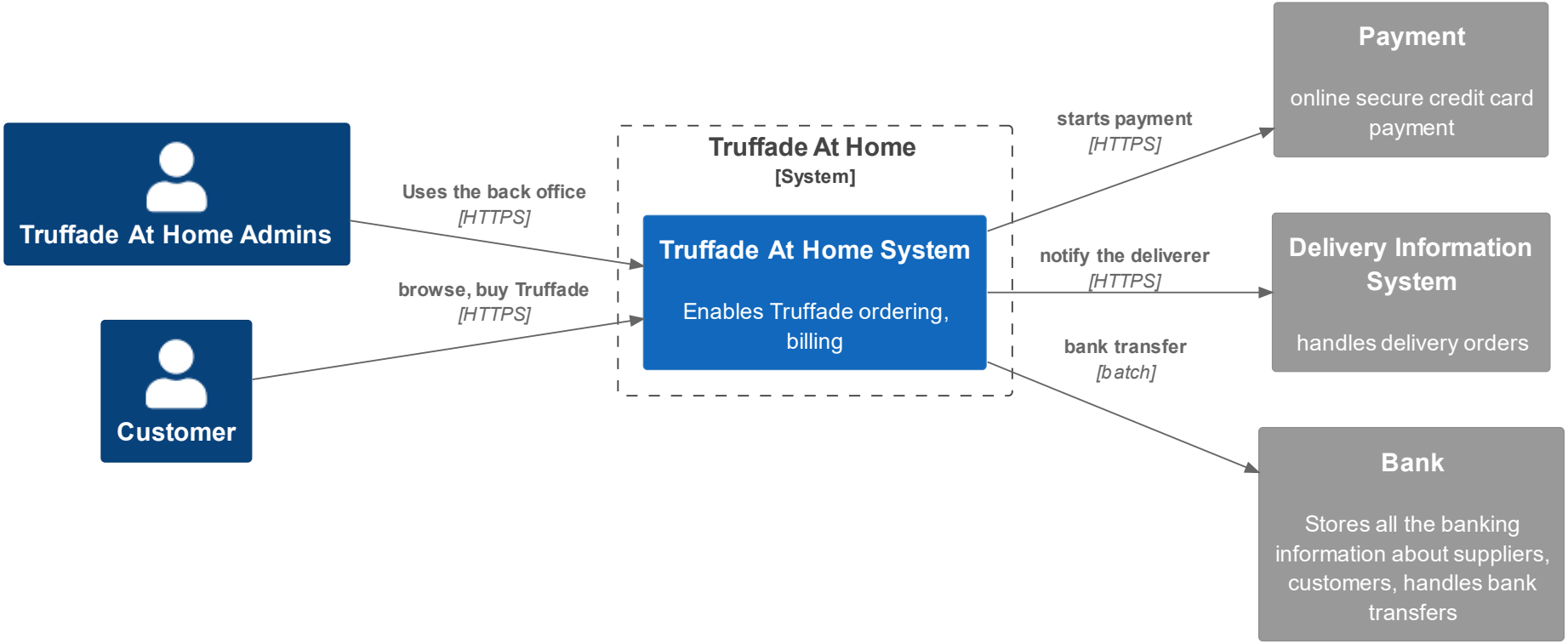


2

Ou celle de **Raphaël** ?

Vue fonctionnelle

La synthèse



Vue fonctionnelle : en résumé



Déclinez l’architecture en plusieurs vues



Confrontez les points de vue



Évitez le syndrome *« Not Invented Here »*

La vue applicative

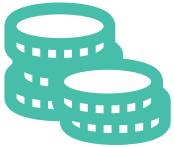
Principales caractéristiques d’une architecture



Évolutivité



Modularité



Coût



Performance



Simplicité

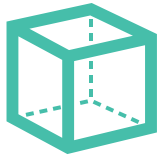


Testabilité



Tolérance
aux pannes

Types d’architecture



Monolithe



SOA



Orchestration

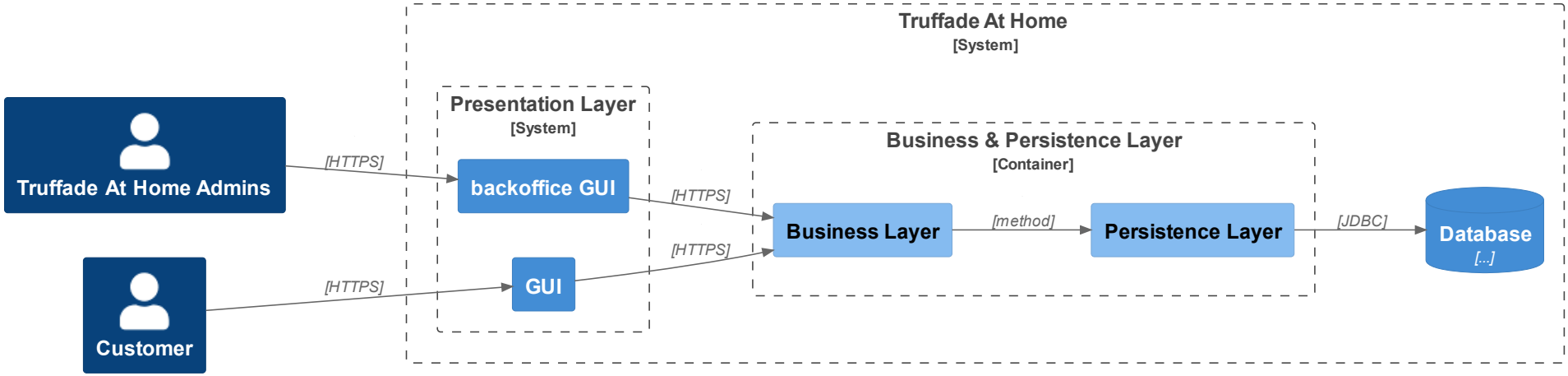


Event Driven

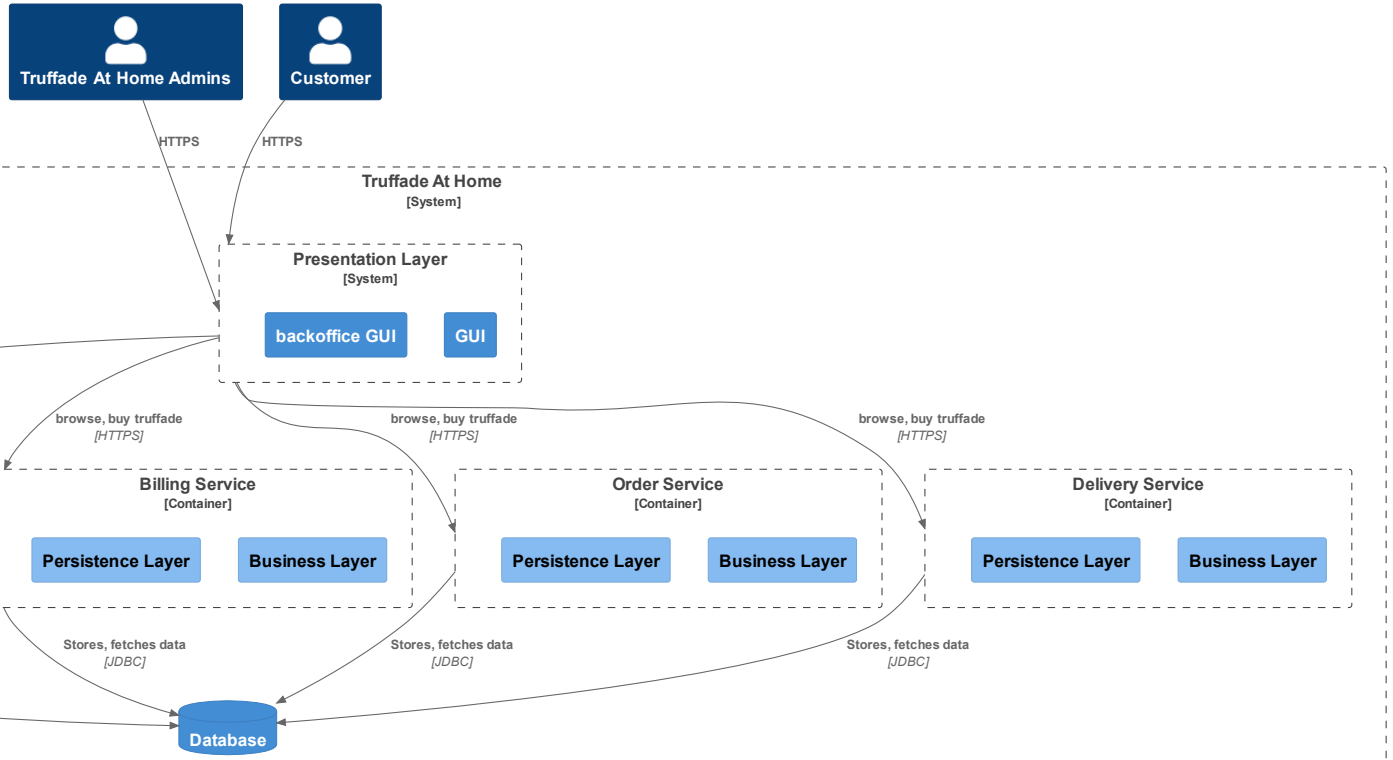


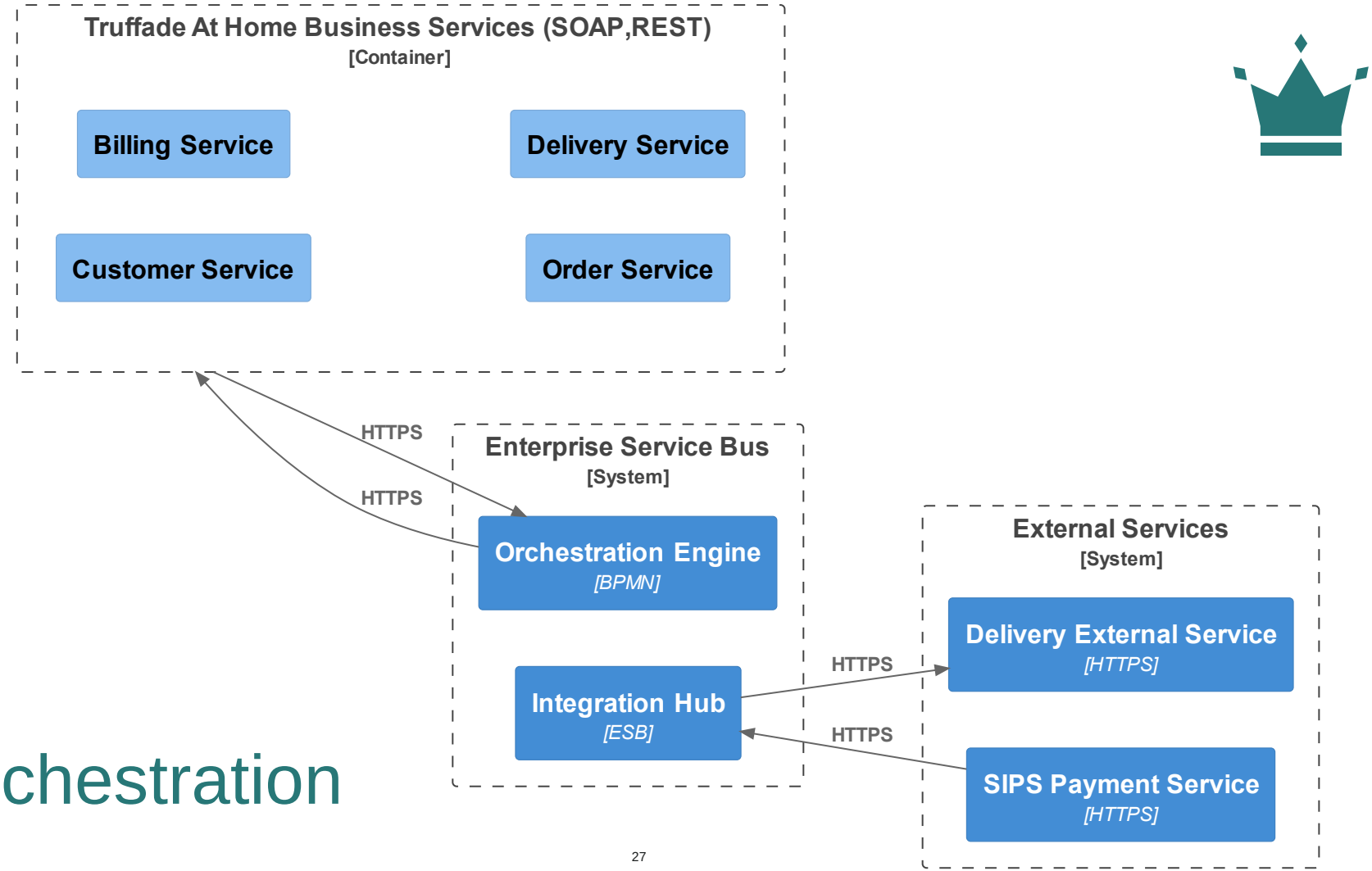
Micro services

Monolithe


Truffade At Home
[System]
Presentation Layer
[System]
Business & Persistence Layer
[Container]
Truffade At Home Admins
Customer
backoffice GUI
GUI
Business Layer
Persistence Layer
Database
[...]
[HTTPS]
[HTTPS]
[HTTPS]
[HTTPS]
[method]
[JDBC]

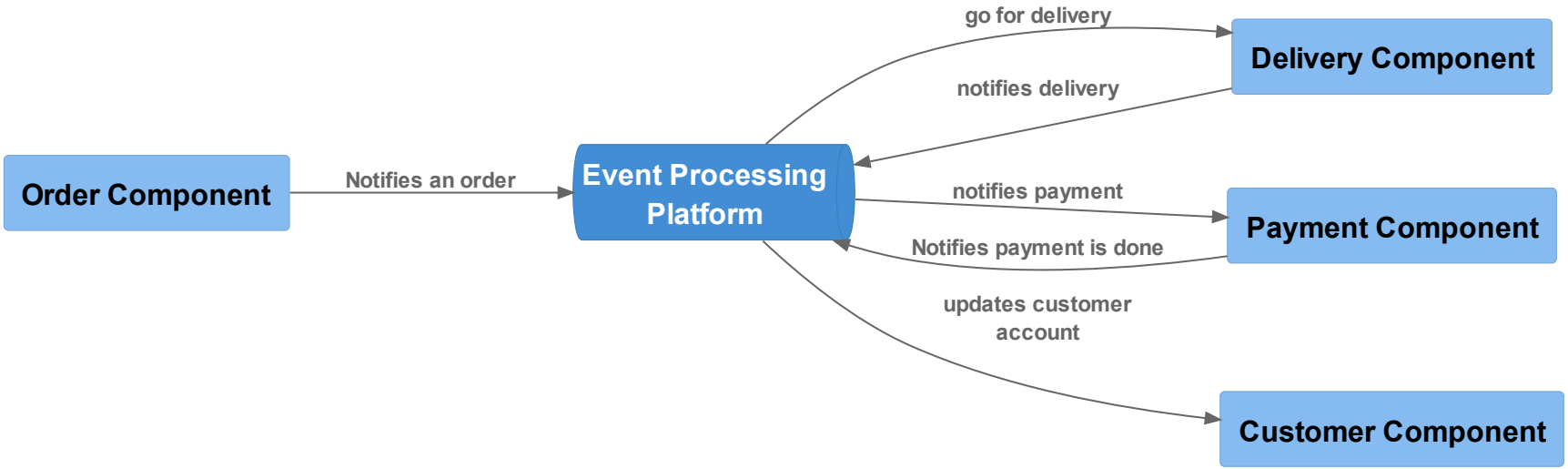
SOA


Truffade At Home Admins
Customer
HTTPS
HTTPS
Truffade At Home
[System]
Presentation Layer
[System]
backoffice GUI
GUI
browse, buy truffade
[HTTPS]
browse, buy truffade
[HTTPS]
browse, buy truffade
[HTTPS]
browse, buy truffade
[HTTPS]
Customer Service
[Container]
Persistence Layer
Business Layer
Billing Service
[Container]
Persistence Layer
Business Layer
Order Service
[Container]
Persistence Layer
Business Layer
Delivery Service
[Container]
Persistence Layer
Business Layer
Stores, fetches data
[JDBC]
Stores, fetches data
[JDBC]
Stores, fetches data
[JDBC]
Stores, fetches data
[JDBC]
Database

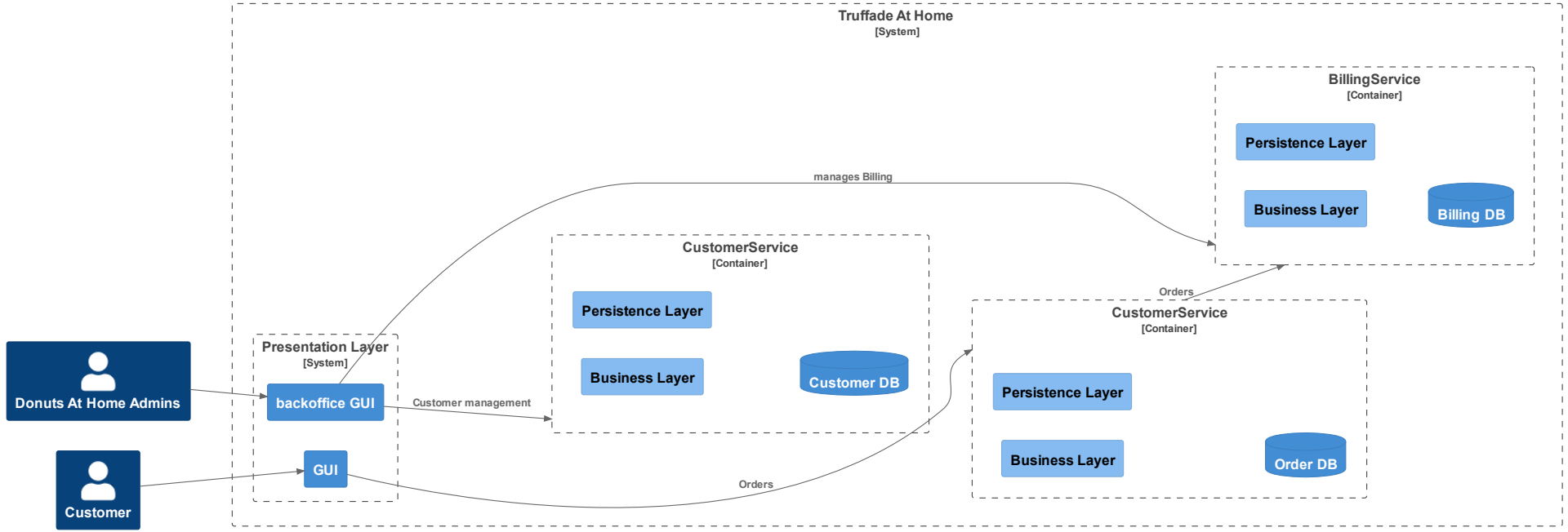


Orchestration

Event Driven

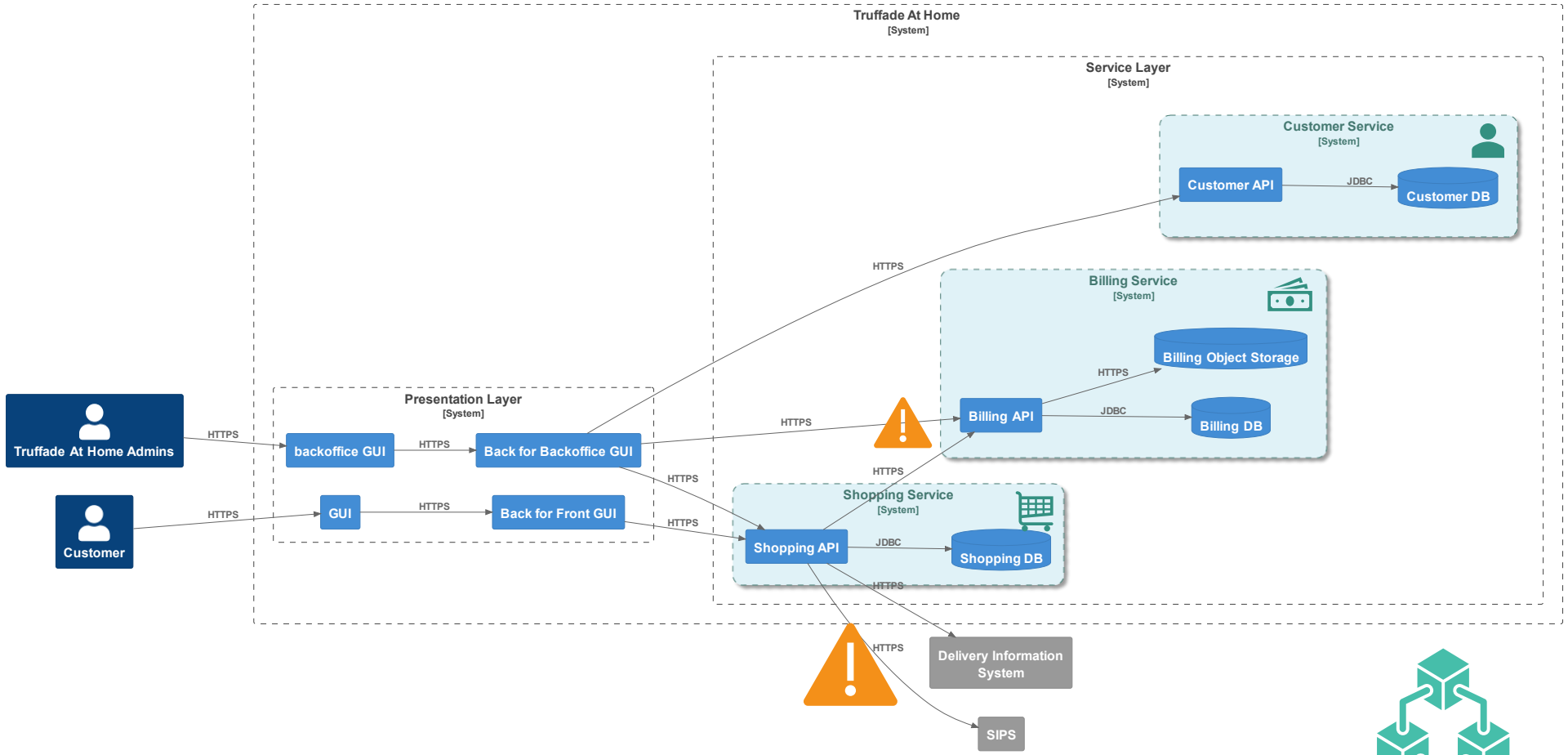


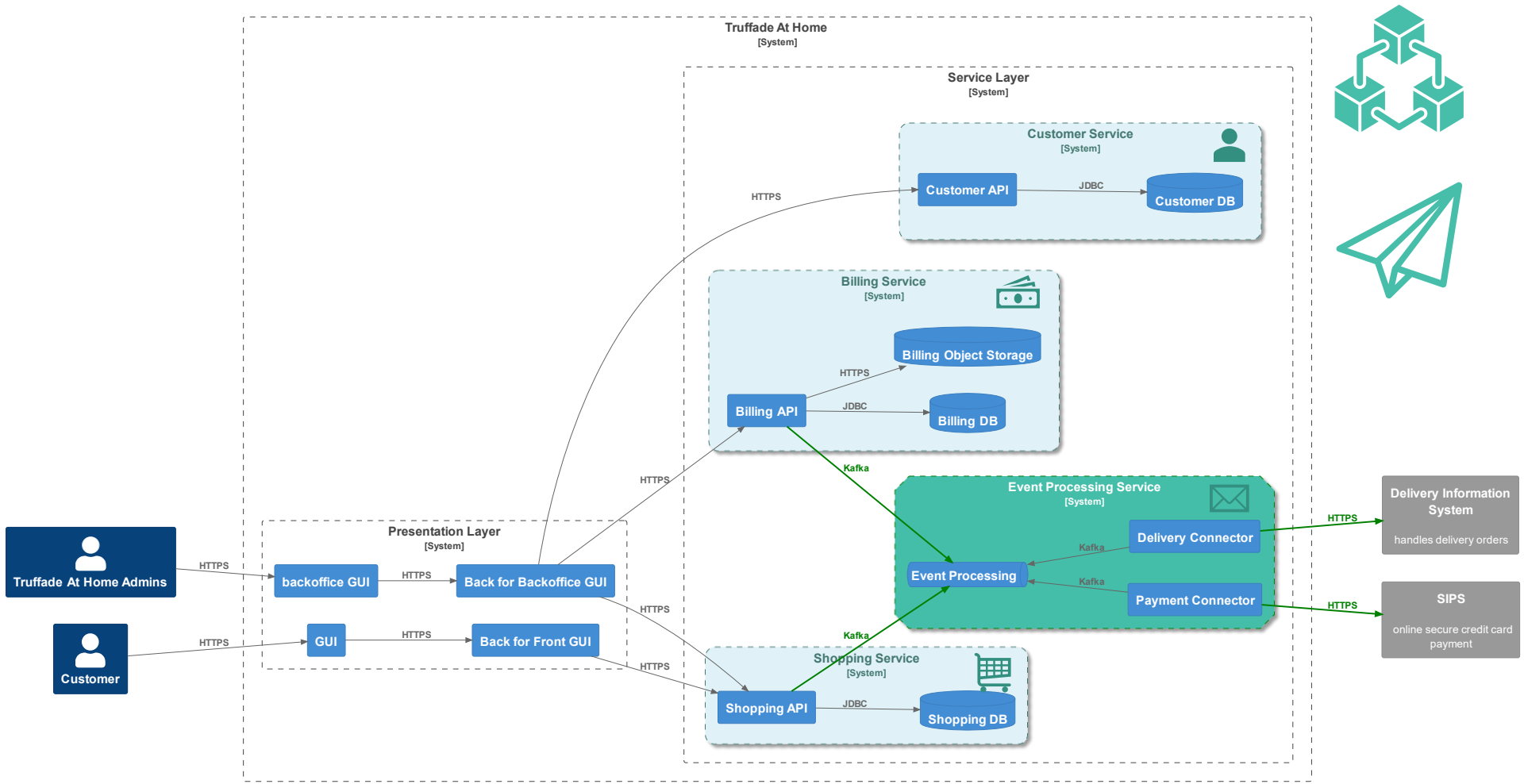
Microservices

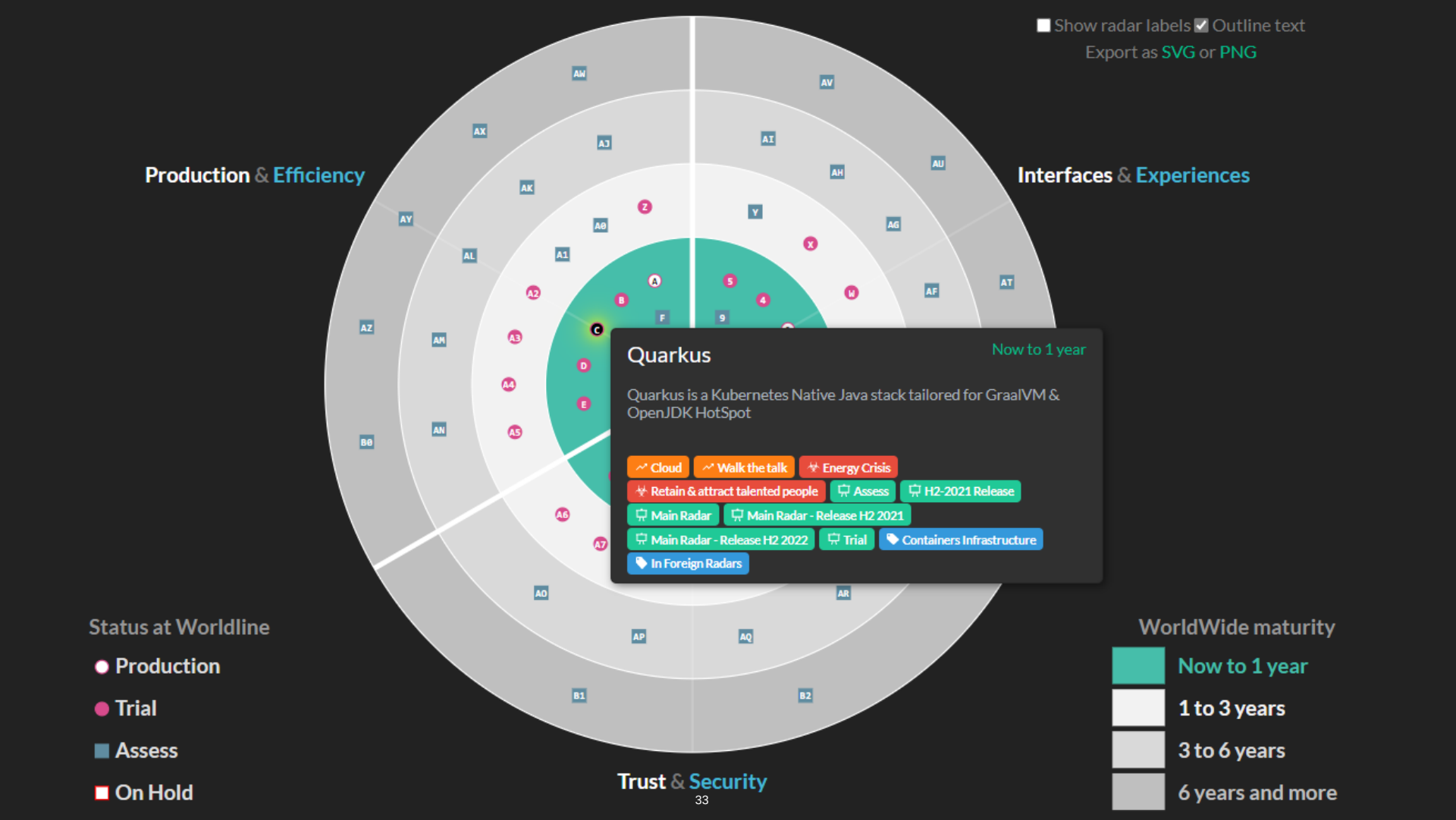


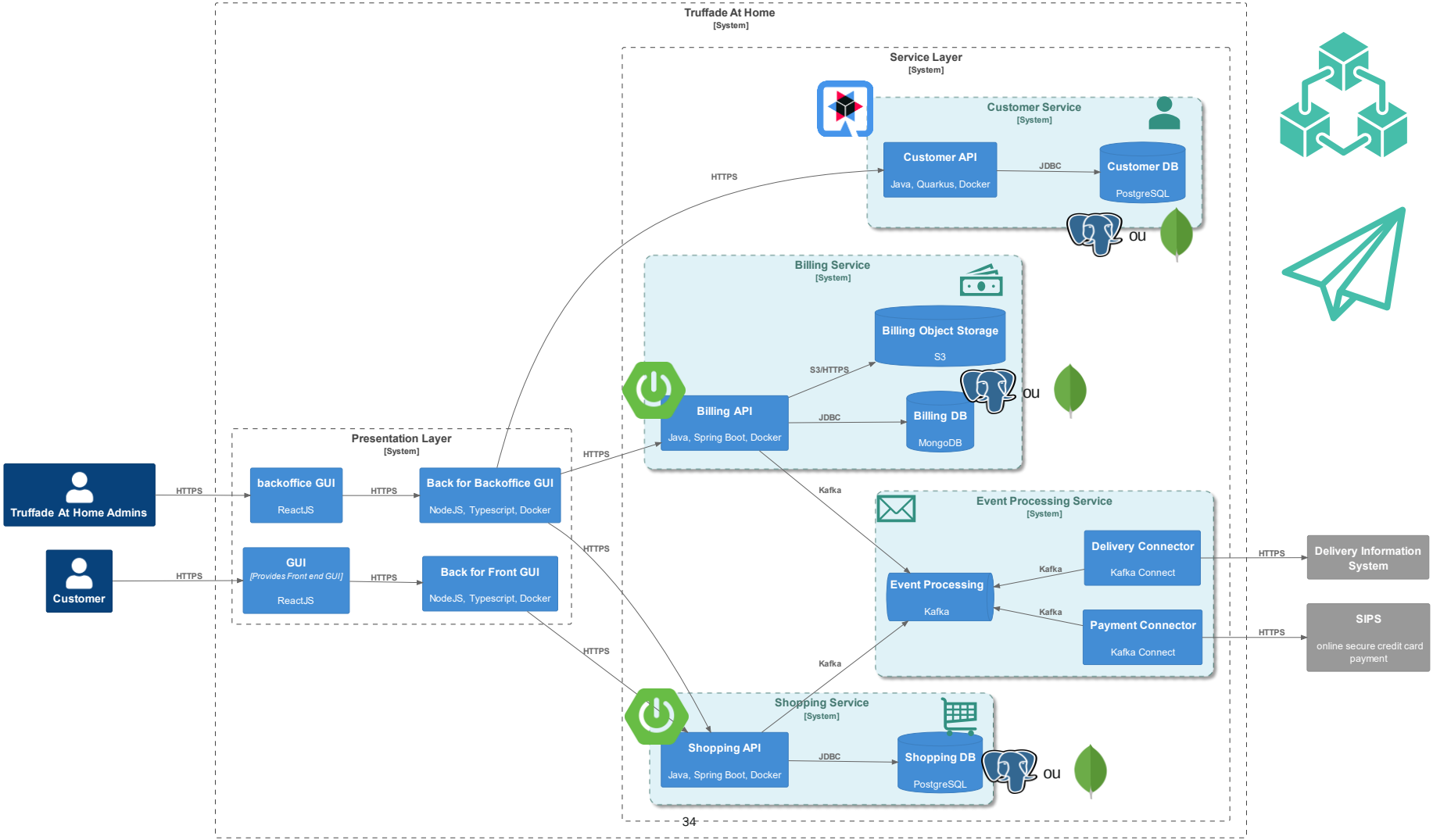
Quand les utiliser ?

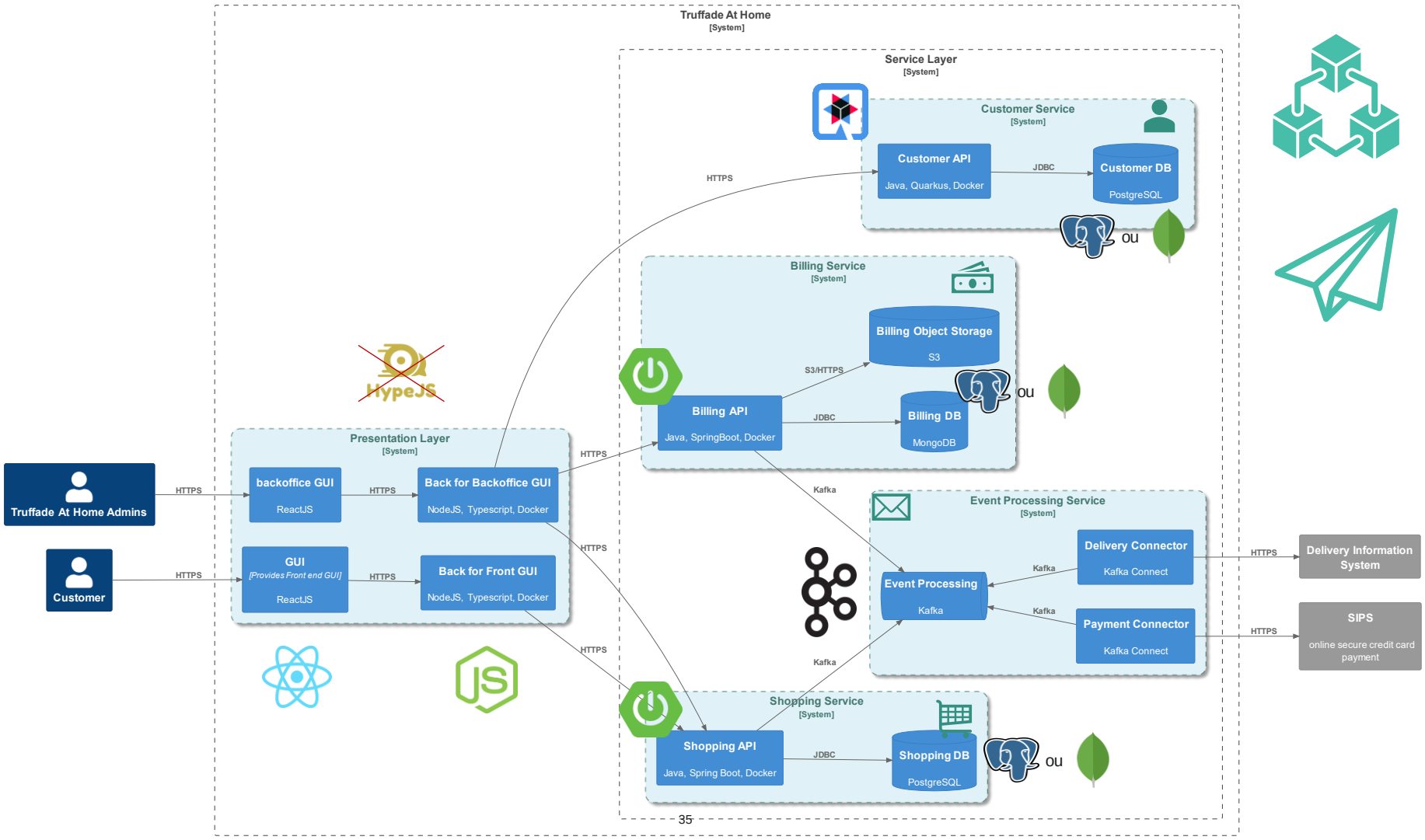
	Monolithe	SOA	Orchestration	Event Driven	Micro-services
Evolutivité	▲▲	▲▲▲	▲	▲▲▲▲▲	▲▲▲▲▲
Scalabilité	▲▲▲	▲▲▲	▲▲▲▲	▲▲▲▲▲	▲▲▲▲▲
Modularité	▲	▲▲▲▲	▲▲▲	▲▲▲▲	▲▲▲▲▲
Coût	▲▲▲▲▲	▲▲▲▲	▲	▲▲▲	▲▲▲
Performance	▲▲	▲▲▲	▲▲	▲▲▲▲▲	▲▲
Simplicité	▲▲▲▲▲	▲▲▲	▲	▲	▲
Testabilité	▲▲	▲▲▲	▲	▲▲	▲▲▲▲
Tolérance aux pannes	▲	▲▲▲▲	▲▲▲	▲▲▲▲▲	▲▲▲▲▲


Truffade At Home
[System]
Service Layer
[System]
Customer Service
[System]
Customer API
JDBC
Customer DB
HTTPS
Billing Service
[System]
Billing Object Storage
HTTPS
Billing API
JDBC
Billing DB
Truffade At Home Admins
HTTPS
Presentation Layer
[System]
backoffice GUI
HTTPS
Back for Backoffice GUI
HTTPS
HTTPS
HTTPS
Shopping Service
[System]
Customer
HTTPS
GUI
HTTPS
Back for Front GUI
HTTPS
Shopping API
JDBC
Shopping DB
HTTPS
HTTPS
Delivery Information System
SIPS


Truffade At Home
[System]
Service Layer
[System]
Customer Service
[System]
Customer API
JDBC
Customer DB
HTTPS
Billing Service
[System]
Billing Object Storage
HTTPS
Billing API
JDBC
Billing DB
Kafka
Event Processing Service
[System]
Delivery Connector
Kafka
Event Processing
Kafka
Payment Connector
Delivery Information System
handles delivery orders
SIPS
online secure credit card payment
Presentation Layer
[System]
Truffade At Home Admins
Customer
backoffice GUI
GUI
Back for Backoffice GUI
Back for Front GUI
Shopping Service
[System]
Shopping API
JDBC
Shopping DB

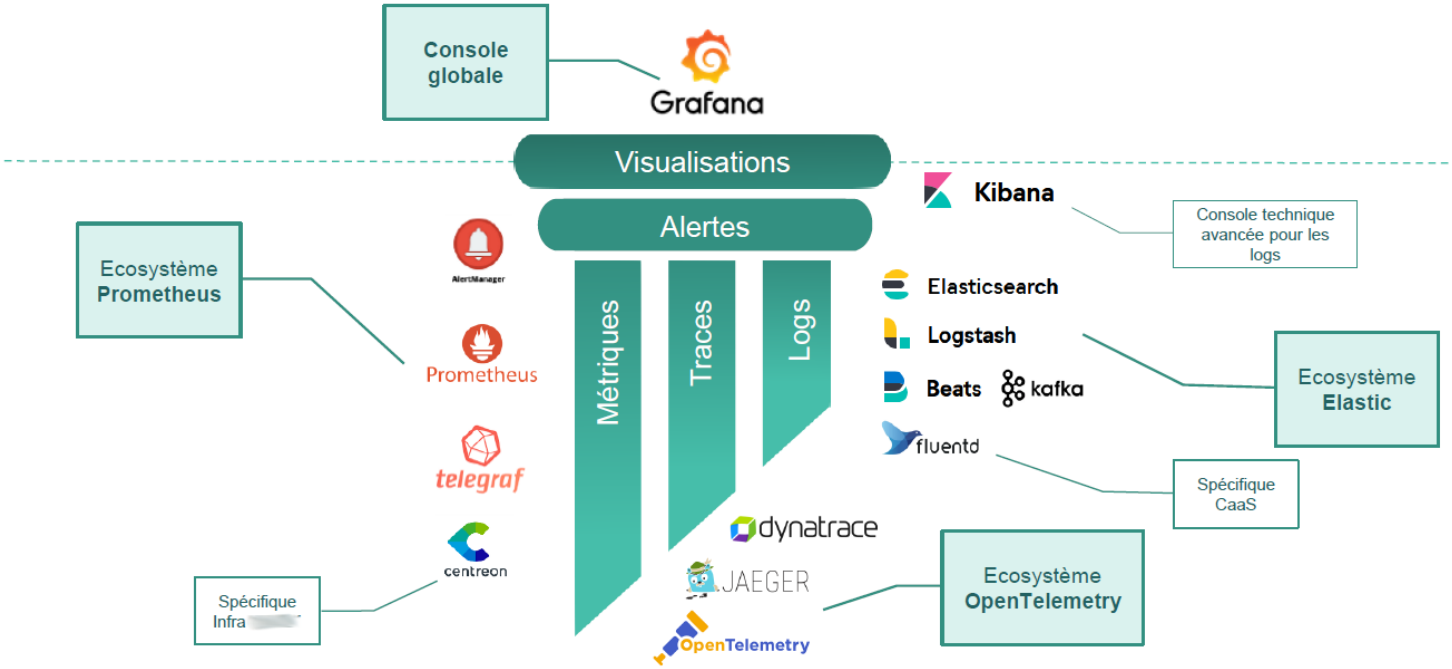

Show radar labels
Outline text
Export as SVG or PNG
Production & Efficiency
Interfaces & Experiences
Trust & Security
Quarkus
Now to 1 year
Quarkus is a Kubernetes Native Java stack tailored for GraalVM & OpenJDK HotSpot
Cloud
Walk the talk
Energy Crisis
Retain & attract talented people
Assess
H2-2021 Release
Main Radar
Main Radar - Release H2 2021
Main Radar - Release H2 2022
Trial
Containers Infrastructure
In Foreign Radars
Status at Worldline
Production
Trial
Assess
On Hold
WorldWide maturity
Now to 1 year
1 to 3 years
3 to 6 years
6 years and more


Truffade At Home
[System]
Service Layer
[System]
Customer Service
[System]
Customer API
Java, Quarkus, Docker
JDBC
Customer DB
PostgreSQL
ou
HTTPS
Billing Service
[System]
Billing Object Storage
S3
S3/HTTPS
Billing API
Java, Spring Boot, Docker
JDBC
Billing DB
MongoDB
ou
Presentation Layer
[System]
Truffade At Home Admins
Customer
HTTPS
backoffice GUI
ReactJS
Back for Backoffice GUI
NodeJS, Typescript, Docker
GUI
[Provides Front end GUI]
ReactJS
Back for Front GUI
NodeJS, Typescript, Docker
Kafka
Event Processing Service
[System]
Event Processing
Kafka
Delivery Connector
Kafka Connect
Payment Connector
Kafka Connect
Delivery Information System
SIPS
online secure credit card payment
Shopping Service
[System]
Shopping API
Java, Spring Boot, Docker
JDBC
Shopping DB
PostgreSQL
ou


Truffade At Home
[System]
Service Layer
[System]
Customer Service
[System]
Customer API
Java, Quarkus, Docker
JDBC
Customer DB
PostgreSQL
ou
HTTPS
Billing Service
[System]
Billing Object Storage
S3
S3/HTTPS
Billing API
Java, SpringBoot, Docker
JDBC
Billing DB
MongoDB
ou
HypeJS
Presentation Layer
[System]
Truffade At Home Admins
Customer
backoffice GUI
ReactJS
GUI
[Provides Front end GUI]
ReactJS
Back for Backoffice GUI
NodeJS, Typescript, Docker
Back for Front GUI
NodeJS, Typescript, Docker
HTTPS
Kafka
Event Processing Service
[System]
Event Processing
Kafka
Delivery Connector
Kafka Connect
Payment Connector
Kafka Connect
Delivery Information System
SIPS
online secure credit card payment
Shopping Service
[System]
Shopping API
Java, Spring Boot, Docker
JDBC
Shopping DB
PostgreSQL
ou

Pensez à l'Observabilité dès la conception !

C'est une **capacité** de la plate-forme plus qu'un assemblage d'outils…



Exemple : architecture d'observabilité sur un gros projet

Probability

Impact

	Low 1	Medium 2	High 3
Low 1			
Medium 2	- Les temps de réponse sont trop élevés (> SLO)	- Indisponibilité des systèmes externes - Plate-forme peu observable	
High 3	- Les middlewares indisponibles - Erreur d'accès à la database - Erreur SAN - Erreur réseau VLAN HS, ou élément réseau HS		

Quelques bonnes pratiques



Analyse de risques
à différents niveaux



Ne restez pas
sur vos acquis !



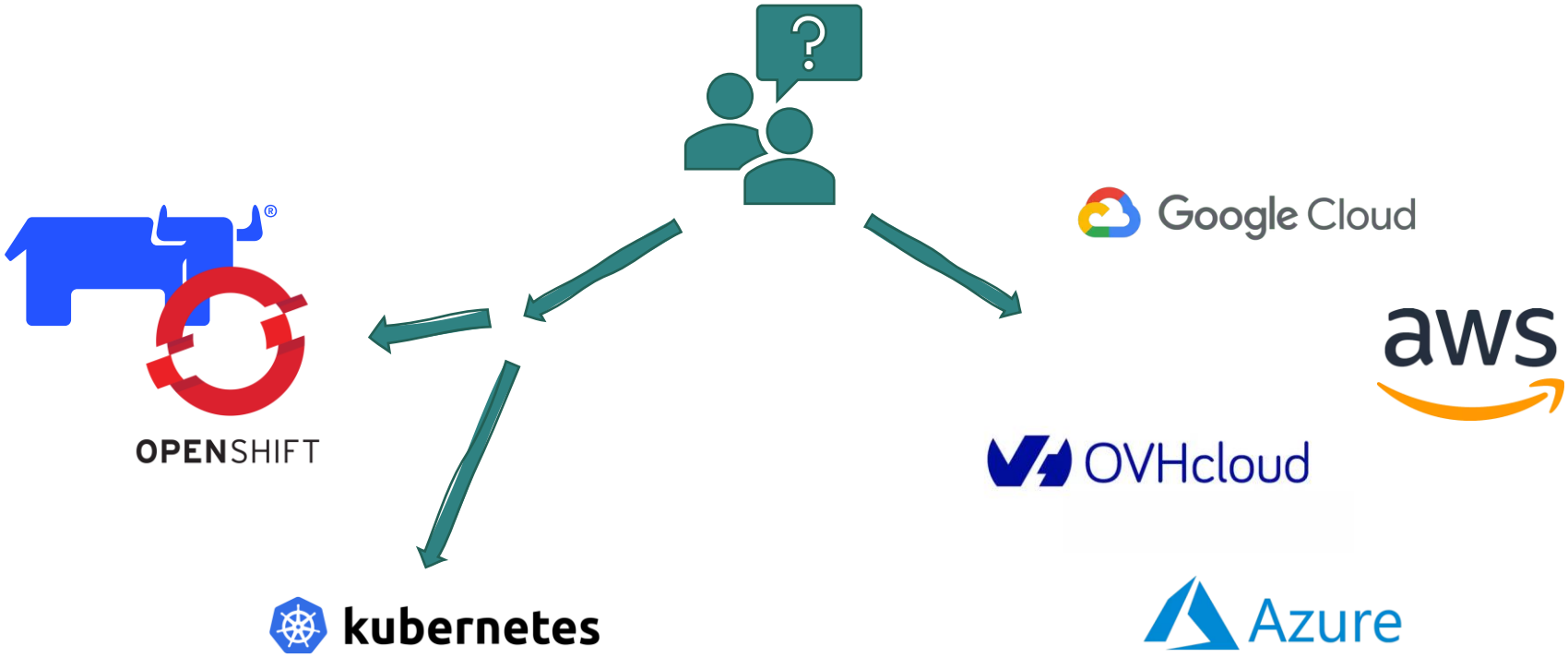
Quand innover ?



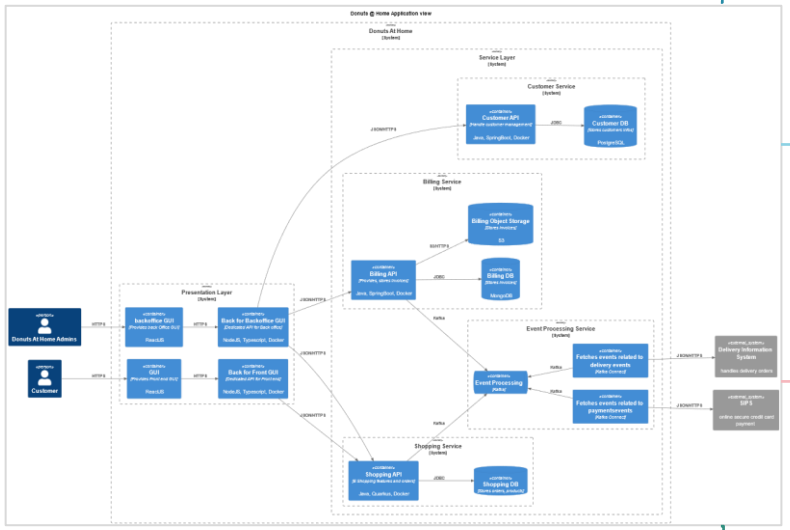
Impacts
organisationnels

L’infrastructure

Le Cloud oui, mais lequel ?



Quels **modes d'utilisation** du Cloud ?



Consommation de **services externes**

Utilisation de **services managés**

Déploiement de **conteneurs**

Déploiement de **machines virtuelles**

SaaS

Paiement …

PaaS

APIM IAM Observabilité

CaaS

Spring Boot Quarkus MongoDB

IaaS

PostgreSQL Kafka

Validation de l’architecture



Conformité aux exigences/contraintes des vues

- **Impliquer** Métier, Devs, Ops, Finance, Experts
- **Formaliser l’engagement** si nécessaire
- Eventuelles **étapes de validation**

Pour l’infrastructure



Vérification de faisabilité ou d’hypothèses techniques (POC) -
Non Functional Requirements -
Eléments de **dimensionnement** -



Travail itératif !

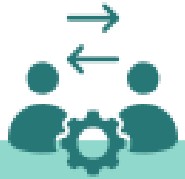
Conclusion

A retenir pour vos ateliers d'architecture



Attitude

- Prendre du recul
- Rester ouvert d'esprit
- Echanger avec tous



Démarche

- Formaliser et tracer
- Collaborer et itérer
- Aligner les vues



Pratiques & Outils

- Patrons d'architecture
- Choix technologiques
- Rester pragmatique

Don’t be a stranger!

Follow & get in touch

Feedback

Follow us:

linkedin.com/in
/raphaelsemeteys

@RaphaelSemeteys

linkedin.com/in
/atouret

@touret_alex

Follow our tech team:

blog.worldline.tech

@WorldlineTech

WORLDLINE

Want to shape how the world pays and get paid?

Explore our jobs in tech:
careers.worldline.com



WORLDLINE